

ADR-2 Selección de estilo Arquitectónico

Situación

Aceptado

Contexto

Durante la presentación del diseño inicial, los evaluadores señalaron sobrecarga funcional en el dashboard y problemas de organización arquitectónica, sugiriendo una división clara de responsabilidades por módulos para reducir el acoplamiento. Adicionalmente, la contraparte del proyecto requiere una solución de bajo costo, lo que limita la complejidad de la infraestructura. El sistema debe soportar una PWA para usuarios base y una plataforma de gestión con diferenciación de permisos para administradores e instituciones.

Decisión

Se recomienda implementar una Arquitectura Basada en Servicios (Service-Based Architecture) con partición por dominio. Esta decisión se fundamenta en la evaluación de las siguientes alternativas:

- **Monolito Modular:** Aunque es de bajo costo, presentaba un alto riesgo de derivar en una "Gran Bola de Lodo" debido a la mezcla de lógicas de administración y operación institucional detectada por los evaluadores.
- **Microservicios:** Fue descartado por su excesiva complejidad operativa y altos costos de implementación y mantenimiento, los cuales no se alinean con las restricciones presupuestarias del cliente.
- **Arquitectura Basada en Servicios (Recomendada):** Se recomienda por ser un enfoque pragmático que permite dividir el sistema en servicios de dominio de grano grueso (ej: Servicio de Reportes, Servicio de Gestión Institucional). Esto permite que la interfaz (PWA) consuma servicios específicos, facilitando la modularidad exigida sin la sobrecarga técnica de los microservicios.

Implicaciones de Interacción y Código

Para resolver la sobrecarga funcional, la implementación seguirá estas reglas estructurales:

- **Particionamiento Físico de Backends:** El sistema se dividirá en múltiples unidades de despliegue independientes (servicios de dominio de grano grueso), cada una atendiendo un flujo de negocio específico (ej: **Servicio de Reportes**, **Servicio de Usuarios**, **Servicio de Gestión**).
- **Interacción mediante Capa de API (Gateway):** Las PWAs (frontend) no llamarán a los servicios directamente. Se implementará una Capa de API (Gateway/Proxy) como punto de entrada único para consolidar la seguridad y redirigir peticiones mediante protocolos REST (HTTP) a los puertos específicos de cada servicio.
- **Persistencia Pragmática:** A diferencia de los microservicios, todos los servicios de dominio consultarán una base de datos monolítica compartida. Esto garantiza la fiabilidad mediante transacciones atómicas, asegurando que los reportes y estadísticas sean siempre una "fuente única de verdad".

- **Estructura de Carpetas (Monorepo):** Se utilizará una organización de carpetas basada en dominios de negocio en lugar de capas técnicas. Cada servicio será un proyecto independiente dentro del repositorio raíz:
 - `/services/sighting-management/`
 - `/services/user-authorization/`
 - `/frontend-pwa/` Cada módulo podrá usar un patrón MVC internamente, pero el límite superior será el dominio.
-

Consecuencias

La adopción de este estilo implica el siguiente análisis de trade-offs:

- **Modularidad y Cohesión (+):** Se logra una separación clara entre las funciones de gobierno (Admin) y las operativas (Institución), resolviendo la observación de sobrecarga funcional mediante servicios con responsabilidades únicas.
 - **Costo y Simplicidad (+):** Al utilizar habitualmente una base de datos compartida, se mantienen los costos de infraestructura bajos y se evita la complejidad de la consistencia de datos distribuida.
 - **Fiabilidad (+):** Permite el uso de transacciones atómicas en la base de datos única, asegurando que los reportes y estadísticas del dashboard se mantengan sincronizados en menos de 5 minutos.
 - **Agilidad de Despliegue (+/-):** Los servicios pueden probarse de forma independiente, aunque habitualmente se despliegan como una unidad o en grupos reducidos, lo que equilibra agilidad y facilidad de operaciones.
-

Cumplimiento

El cumplimiento de esta propuesta se medirá mediante:

- **Verificación de Cohesión:** Comprobación de que no existan dependencias directas de código entre la lógica de moderación de reportes (Admin) y el seguimiento operativo (Institución) en la capa de servicios.
 - **Pruebas de Integración:** Validación de que la PWA del usuario base y la plataforma de gestión consuman los servicios de dominio a través de contratos de API (REST) bien definidos.
-

Notas

Autor: @Valentina Cifuentes Poblete

Día de publicación: 23-05-2026

Última actualización: 23-05-2026

